

MyWebApp template

Contents

1. Overview	5
1.1. Technology Stack	7
1.2. Architecture	8
1.3. What's New in v0.8.1	8
1.4. What's New in v0.8.0	9
1.5. What's New in v0.7.1	9
1.6. What's New in v0.7.0	10
1.7. What's New in v0.6.x	10
1.8. Earlier releases	10
2. Prerequisites	11
2.1. Required Software	11
2.2. Install Development Tools	11
2.3. Environment Setup	12
2.4. Secrets Management in Dev and in Prod	16
3. Initial Setup	17
3.1. 1. Clone the Repository	17
3.2. 2. Install Dependencies	17
3.3. 3. Create the Database	17
3.4. 4. Run Migrations	17
3.5. 5. Seed Demo Users	18
3.6. 6. Generate SQLC Code	18

3.7. 7. Build Tailwind CSS	18
3.8. 8. Run the Application	18
3.9. Quick Start (All-in-One)	20
4. Testing	21
4.1. Running the tests	21
4.2. How the test database works	21
4.3. Safety guards	21
4.4. Test coverage	22
5. Configuration Reference	23
5.1. Logging	23
5.2. Rate Limiting	26
5.3. Gzip Compression	26
5.4. HTTP Security Headers	27
5.5. Secure Cookies	28
5.6. CSRF Protection	28
5.7. Authentication	29
5.8. Authorization	31
6. Versioning	32
6.1. Build-time injection	32
6.2. Runtime query	32
6.3. Release procedure	33
7. Common Development Workflow	34
7.1. Daily Development	34
7.2. Database Changes	34
7.3. Adding New Queries	36
7.4. Adding a New API Endpoint	38

7.5. Adding a New HTMX Page	41
8. Project Structure	43
9. Project inventory for v0.8.1	48
10. Available Make Commands	50
10.1. Application	50
10.2. Dependencies	50
10.3. Database	51
10.4. Test database	51
10.5. Testing & Quality	51
10.6. Docker	52
10.7. Setup & Utilities	52
11. API Endpoints	54
11.1. Authentication API	54
11.2. Users API	54
11.3. Web (HTMX) Routes	54
11.4. Operational Endpoints	56
11.5. Example Requests	57
12. Troubleshooting	62
12.1. Common Issues	62
12.2. Getting Help	68
13. Contributing	68
13.1. Conventions	69
14. License	69
15. Contact	69

patrick toca v0.8.1, October 5th, 2026

Creation Date: September 6th, 2026

Author: patrick toca

Email: pmjtoca@gmail.com

1. Overview

MyWebApp is a modern web application built with Go, using a clean architecture with PostgreSQL as the database. It combines a server-rendered HTMX frontend with a JSON API, sharing the same service layer.

The project leverages powerful tools like Gin for HTTP routing, SQLC for type-safe database queries, and `log/slog` for structured logging.

The application ships with a **public home page** reachable without authentication, a **read-only member directory** available to any authenticated user, and an **admin-only user-management surface** for creating and modifying accounts. The binary reports its own build metadata (version, commit, branch, build date) at runtime. A separate **test database environment** runs every DB-dependent test against an isolated PostgreSQL instance.

MyWebApp — sequence diagram

Anonymous visitor loads the Home page, signs in, and loads the admin user list.

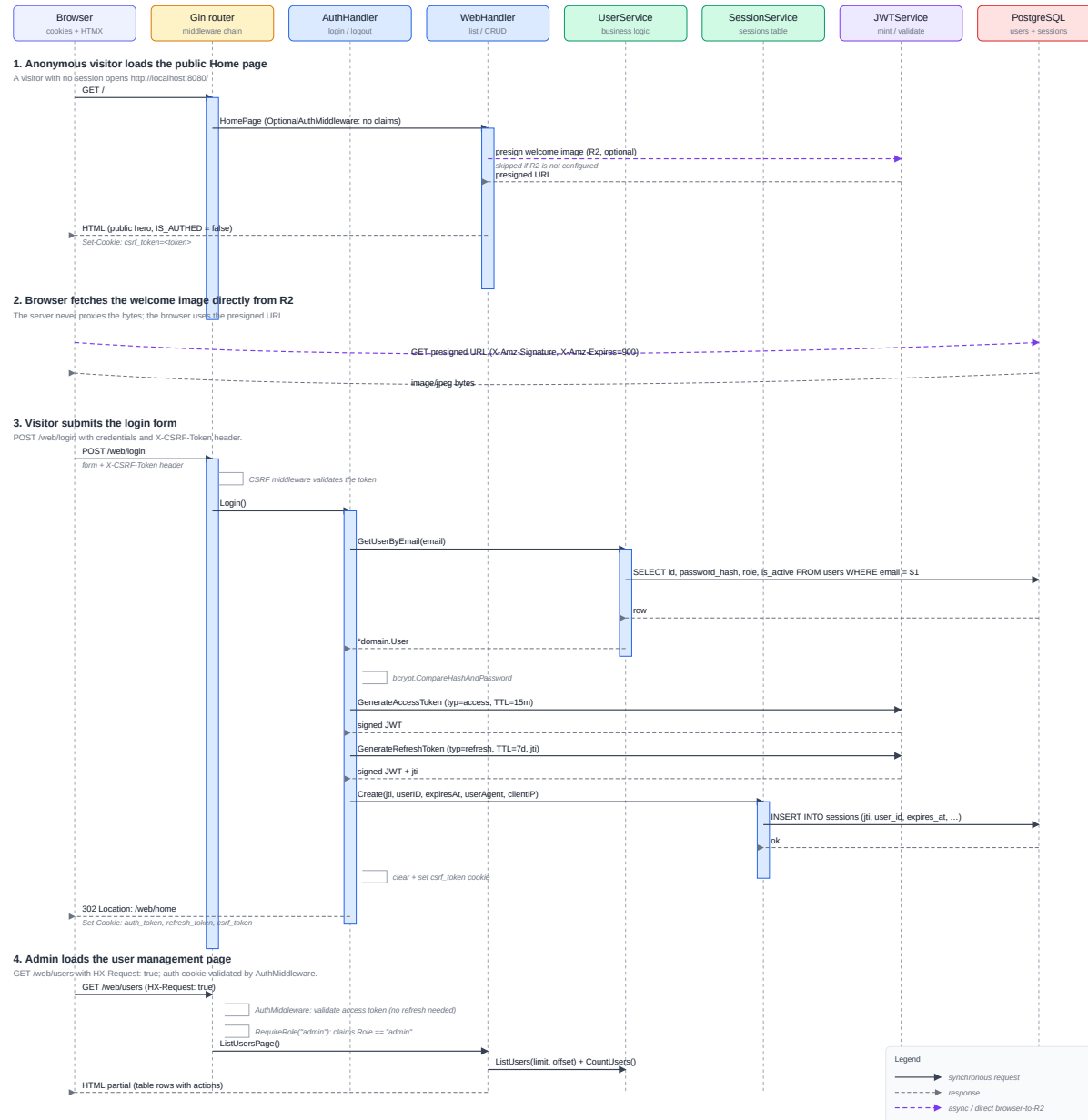


Figure 1. Login and admin-list sequence

1.1. Technology Stack

- **Go 1.21+** — Primary programming language
- **Gin** — HTTP web framework (with custom middleware for gzip, rate limiting, structured logging, security headers, and CSRF)
- **HTMX** — Hypermedia-driven frontend (no build step for interactivity)
- **Tailwind CSS** — Utility-first CSS framework
- **PostgreSQL** — Database
- **pgx/v5** — PostgreSQL driver and connection pool
- **SQLC** — Type-safe SQL code generation
- **log/slog** — Structured logging (JSON or text)
- **JWT** ([golang-jwt/jwt/v5](#)) — Access and refresh tokens with server-side revocation
- **bcrypt** ([golang.org/x/crypto](#)) — Password hashing
- **Cloudflare R2** (optional) — Private welcome image via presigned URLs
- **Air** — Live reload for development

1.2. Architecture

The application follows a clean architecture pattern:

- **Domain Layer** (`internal/domain/`) — Core business models, DTOs, and domain errors
- **Service Layer** (`internal/service/`) — Business logic, transaction orchestration, session management
- **Repository Layer** (`internal/repository/`) — Data access (SQLC-generated)
- **Handler Layer** (`internal/handler/`) — JSON API handlers
- **Web Layer** (`internal/web/`) — HTMX/HTML handlers, auth flows, cookie helpers, template helpers
- **Middleware Layer** (`internal/middleware/`) — Request ID, access log, gzip, rate limiting, security headers, CSRF, role check
- **Logging Layer** (`internal/logging/`) — Process-wide `slog` configuration
- **Auth Layer** (`internal/auth/`) — JWT issuance/validation, password hashing
- **Version Layer** (`internal/version/`) — Build-time metadata injected by the linker
- **Testutil Layer** (`internal/testutil/`) — Isolated test database harness
- **Database Layer** (`pkg/db/`) — Pool configuration, transactions, `.env` loading
- **Object Storage** (`pkg/r2/`) — Cloudflare R2 client for presigned URLs

1.3. What's New in v0.8.1

- **CSRF token signing** — the CSRF cookie now carries an HMAC signature over the token body. The middleware rejects any cookie whose signature does not verify, closing the cookie-tossing path in which an attacker plants a valid-looking cookie.
- **Bearer exemption tightened** — `CSRFMiddleware` skips validation only when `AuthMiddleware` has explicitly marked the request as Bearer-authenticated. A request that merely carries an `Authorization` header is no longer exempt.
- **API paths accept only Bearer tokens** — cookie authentication is refused on `/api/*`. The API group has no CSRF middleware, so accepting cookies there would expose the JSON endpoints to CSRF.
- **Bearer token endpoint** — `POST /api/v1/auth/token` exchanges credentials for a Bearer access token, giving API clients a supported way to

authenticate.

- **Silent refresh reloads the user** — the middleware now reads the current role and `is_active` flag from the database on every refresh, instead of trusting the stale claims in the refresh token.
- **Logout revokes the correct session** — `Logout` parses the refresh cookie to obtain the `jti` before revoking. The previous implementation called `Revoke` with an empty `jti`, which was a no-op.
- **Rate limiter hardening** — keying now uses Gin's `ClientIP()`, which honours the trusted-proxy configuration; IPv6 is keyed by `/64`; the per-limiter map is capped and falls back to a shared bucket when full; `Retry-After` reflects the limiter's actual rate; a `Stop` method prevents the cleanup goroutine from leaking.

1.4. What's New in v0.8.0

- **Isolated test database environment** — `internal/testutil` provides a per-binary test harness that resets the schema, applies migrations in-process, and seeds the demo users. Tests read `TEST_DB_*` environment variables and never touch the production database; `db.IsTestConfig` refuses to run if the two names coincide.
- **Expanded test coverage** — source-only coverage rose from 20.5% to 42.3%. The packages at or above the target are `internal/auth` (78%), `internal/handler` (62%), `internal/middleware` (88%), `internal/service` (60%), and `internal/version` (66%).
- **Bug fixes caught by the new tests** — `UserService.DeleteUser` now returns `ErrUserNotFound` for a nonexistent user instead of silently succeeding; the gzip middleware no longer advertises compression on 204 responses.

1.5. What's New in v0.7.1

- **Build-time version metadata** — `internal/version` carries the semantic version, git commit, branch, build date, and dirty flag, injected by `-ldflags -X` at build time. A new public `GET /version` endpoint reports the same information as JSON.
- **Release build tooling** — `scripts/build.sh` compiles with the correct `-ldflags` and `-trimpath` flags. The `make build` target uses the same flags and prints the metadata before compiling.
- **Documentation** — this README and `docs/security_headers_cookies_csrf_jwt_scheme.adoc` updated for the v0.7.1 public home page, member directory, and version endpoint.
- **Stats endpoint available to all roles** — `/web/users/stats` moved out of the admin-only group so the Home page can display member

counts to every authenticated user, not only admins.

1.6. What's New in v0.7.0

- **Public home page** — `/` and `/web/home` are served to anonymous visitors and authenticated users alike. The page renders a public hero for visitors and a personalised one for signed-in users, using `OptionalAuthMiddleware` so no redirect occurs for anonymous callers.
- **Read-only member directory** — `/web/members` lists registered users (all roles: user, moderator, admin) to every authenticated caller. Mutation stays admin-only under `/web/users`.
- **Unified post-login redirect** — every authenticated role now lands on `/web/home` after login or registration. The previous role-dependent redirect (admin → `/web/users`, others → `/web/me`) has been removed.
- **Cache-Control hardening** — `abortUnauthenticated` now emits `Cache-Control: no-store` on every 401 and login redirect, so browsers cannot heuristically cache a redirect that survives a redeploy.

1.7. What's New in v0.6.x

- **Role-based authorization** — `RequireRole` middleware, admin-only route groups, and dedicated role-change endpoints.
- **Self-service landing page** — `/web/me` for non-admin users.
- **Private R2 welcome image** — presigned URLs generated on demand.
- **Dedicated role-change forms** — separate DTO (`UpdateUserRoleRequest`) keeps `role` out of create/update payloads.

1.8. Earlier releases

- **v0.5.0** — HTTP security headers, secure cookies, CSRF protection, two-token JWT scheme with a `sessions` table, silent refresh.
- **v0.4.0** — HTTP security headers and secure cookie attributes.
- **v0.3.0** — Structured logging, request IDs, gzip, rate limiting, graceful shutdown, health endpoint.

2. Prerequisites

2.1. Required Software

- Go 1.21 or higher
- PostgreSQL 14 or higher
- Node.js 18+ (for Tailwind CSS build)
- Make (optional but recommended)
- Git

2.2. Install Development Tools

```
# Install Air (live reload)
go install github.com/air-verse/air@latest

# Install SQLC (type-safe SQL)
go install github.com/sqlc-dev/sqlc/cmd/sqlc@latest

# Install golang-migrate (migrations), with the PostgreSQL driver compiled in
go install -tags 'postgres' github.com/golang-migrate/migrate/v4/cmd/migrate@latest
```



The `-tags 'postgres'` flag is required. Without it, the `migrate` binary has no database drivers and will fail with `unknown driver postgres (forgotten import?)` on first use.

2.3. Environment Setup

Create a `.env` file in the project root:

```
# Server
PORT=8080
GIN_MODE=release

# Database
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=postgres
DB_NAME=mywebapp
DB_SSLMODE=disable

# Pool tuning (optional – sensible defaults apply)
DB_MAX_CONNS=25
DB_MIN_CONNS=5
DB_MAX_CONN_LIFETIME=1h
DB_MAX_CONN_IDLE_TIME=30m
DB_HEALTH_CHECK_PERIOD=1m
DB_CONNECT_TIMEOUT=5s
DB_STATEMENT_TIMEOUT=30s

# -----
# Test database
# -----
# Tests run against a separate database, never against DB_NAME above.
# TEST_DB_NAME must not equal DB_NAME. If it is empty or matches,
# every DB-dependent test skips with a clear message.
#
# The test database is created and migrated automatically on the first
```

```
# `make test`.
TEST_DB_HOST=localhost
TEST_DB_PORT=5432
TEST_DB_USER=postgres
TEST_DB_PASSWORD=postgres
TEST_DB_NAME=mywebapp_test
TEST_DB_SSLMODE=disable

# Pool tuning for the test database (smaller than production).
TEST_DB_MAX_CONNS=10
TEST_DB_MIN_CONNS=1
TEST_DB_CONNECT_TIMEOUT=5s
TEST_DB_STATEMENT_TIMEOUT=30s

# Auth – ALWAYS override in production
JWT_SECRET=change-me-in-production-please

# JWT token TTLs
# Access: short (the credential presented on every request)
JWT_ACCESS_TTL=15m
# Refresh: long (keeps the user logged in across browser sessions)
# Capped at 720h (30 days) by the parser.
JWT_REFRESH_TTL=168h

# Logging
LOG_FORMAT=json
LOG_LEVEL=info

# Security headers
CSP_REPORT_ONLY=true      # set to false in production, after a clean browser check
CSP_REPORT_URI=           # optional endpoint for violation reports
HSTS_MAX_AGE=0           # set to 31536000 ONLY after HTTPS is verified
HSTS_INCLUDE_SUBDOMAINS=false
HSTS_PRELOAD=false
```

```
X_FRAME_OPTIONS=DENY
REFERRER_POLICY=strict-origin-when-cross-origin
PERMISSIONS_POLICY=          # empty = middleware's built-in default
COOP=same-origin
CORP=same-origin

# Secure cookies
COOKIE_SECURE=false          # set to true in production, ONLY over HTTPS
COOKIE_SAMESITE=lax
COOKIE_DOMAIN=               # empty = host-only (correct for localhost)

# CSRF (defaults shown; override only if you have a reason)
CSRF_FORM_FIELD=csrf_token
CSRF_HEADER_NAME=X-CSRF-Token
CSRF_TOKEN_LENGTH=32
CSRF_COOKIE_MAX_AGE=0
# CSRF_SECRET signs the CSRF token. Required in a cluster so all nodes
# can verify tokens issued by any of them. If unset, a random
# per-process key is generated (fine for single-instance deployments,
# but a restart invalidates outstanding tokens).
CSRF_SECRET=

# Cloudflare R2 (optional – leave all four empty to disable the welcome image)
R2_ACCOUNT_ID=
R2_ACCESS_KEY_ID=
R2_SECRET_ACCESS_KEY=
R2_BUCKET_NAME=
R2_IMAGE_KEY=welcome.jpg
```



For local HTTP development, `COOKIE_SECURE=false` is mandatory. If left at the `true` default while running over plain HTTP, the browser accepts the cookie on login but silently refuses to send it back — you will appear logged in for one response and then logged out.

The test database is created and migrated by the test suite itself, on the first `make test` of a fresh clone. `internal/testutil` resets the schema, runs the migrations in-process, and seeds the demo users. `db.IsTestConfig` refuses to run if `TEST_DB_NAME` equals `DB_NAME` or does not look like a test database, so the suite cannot accidentally wipe production.

2.4. Secrets Management in Dev and in Prod

See [the age/SOPS/Fly how-to](#) for secret management.

3. Initial Setup

3.1. 1. Clone the Repository

```
git clone https://github.com/yourusername/mywebapp.git  
cd mywebapp
```

3.2. 2. Install Dependencies

```
make install  
# or manually:  
go mod download  
go mod tidy  
npm install
```

3.3. 3. Create the Database

```
createdb mywebapp  
# or:  
psql -c "CREATE DATABASE mywebapp;"
```

3.4. 4. Run Migrations

```
make migrate-up
```

This creates the `users` and `sessions` tables, along with their indexes and the foreign key from `sessions.user_id` to `users.id`.

3.5. 5. Seed Demo Users

```
make seed
```

This inserts five bcrypt-hashed demo users. The primary demo login is:

- **alice@example.com** / `password123` (admin)

3.6. 6. Generate SQLC Code

```
make generate
```

Regenerates `internal/repository/` from `sqlc/queries.sql` and `sqlc/queries_sessions.sql`.

3.7. 7. Build Tailwind CSS

```
make build-css
```

3.8. 8. Run the Application

```
# Run normally
make run

# Or with live reload (recommended for development)
```

```
make dev
```

3.9. Quick Start (All-in-One)

```
make setup && make seed && make dev
```

This will:

1. Install all dependencies
2. Generate SQLC code
3. Run migrations
4. Build Tailwind CSS
5. Seed demo users
6. Start the application with live reload

4. Testing

The project ships with an isolated test database environment. Tests never touch the production database.

4.1. Running the tests

```
make test           # full suite, serialised (-p 1)
make test-coverage  # HTML report over every package
make test-coverage-src # report excluding generated/wiring packages
```

`make test` runs every package in sequence (`-p 1`) so that the shared test database is not accessed by two packages at once.

4.2. How the test database works

`internal/testutil` is the harness. Its `Setup()` function, called from each DB-dependent package's `TestMain`, does four things:

1. Opens a pool to the database named by `TEST_DB_*`.
2. Resets the schema (`DROP SCHEMA public CASCADE; CREATE SCHEMA public`).
3. Runs the migrations in-process, using the same migration files that `make migrate-up` applies to production.
4. Seeds the demo users so tests can rely on `alice@example.com` existing with a known password.

The pool is shared across all tests in a package and closed once, after the last test runs. Individual tests must not close it.

4.3. Safety guards

Three properties prevent the tests from touching production:

- `TEST_DB_*` are separate variables from `DB_*`. A mistake in one cannot affect the other.

- `db.IsTestConfig` refuses to run if `TEST_DB_NAME` equals `DB_NAME`.
- If `TEST_DB_NAME` is empty or does not contain `test`, `ci`, `tmp`, or `scratch`, `LoadTestConfig` returns `(nil, false)` and every DB-dependent test skips with a clear message.

4.4. Test coverage

Source-only coverage (excluding `cmd`, `scripts`, `domain`, `repository`, `logging`, and `testutil`) is approximately 58%. The per-package figures:

Package	Coverage
<code>internal/auth</code>	~78%
<code>internal/handler</code>	~62%
<code>internal/middleware</code>	~88%
<code>internal/service</code>	~60%
<code>internal/version</code>	~66%
<code>internal/web</code>	~70%

To see the exact numbers on your machine:

```
make test-coverage-src
```

That target writes `coverage-src.html` and prints the total to the terminal. Open the HTML file in a browser to see which lines are covered and which are not.

5. Configuration Reference

5.1. Logging

Logging is configured entirely through environment variables. Two formats are supported:

Variable	Values	Description
LOG_FORMAT	text json	Output format. Default: <code>text</code> when stderr is a TTY, <code>json</code> when redirected (good for production log shippers).
LOG_LEVEL	debug info warn error	Minimum level. <code>debug</code> also enables source location (<code>AddSource</code>).

Logs are written to `logs/app.log`. When running in an interactive terminal, they are also mirrored to stderr.

5.1.1. Structured Log Fields

Every HTTP request produces one access log line with:

```
{
  "time": "2026-10-02T12:34:56.214266491+01:00",
  "level": "INFO",
  "msg": "http_request",
  "request_id": "a9346c63f1de4251ddf025d9043d453d",
  "method": "POST",
  "path": "/web/login",
  "status": 200,
  "latency": 2781319,
  "client_ip": "::1",
```

```
"size": 2249  
}
```

5.1.2. Request Correlation

Every request is assigned a `request_id` (or honours an inbound `X-Request-ID` header). The ID is:

- Returned to the client as the `X-Request-ID` response header.
- Attached to the request-scoped `*slog.Logger`.
- Included on every line emitted through `middleware.LoggerFromGin(c)`.

Example — find every log line for one request:

```
grep '"request_id":"a9346c63f1de4251ddf025d9043d453d"' logs/app.log
```

5.2. Rate Limiting

Three token-bucket limiters run by default:

Scope	Rate	Burst	TTL
Global (all routes)	20 req/s	40	10 min
Auth routes (<code>/web/login</code> , <code>/web/register</code> , <code>/api/v1/auth/token</code>)	5 req/min	5	15 min
Refresh route (<code>/web/refresh</code>)	1 req/s	30	15 min

Buckets are keyed by the client IP returned from Gin's `ClientIP()`, which honours the trusted-proxy configuration set on the engine (`router.SetTrustedProxies`). The application sets this at startup based on the deployment's network topology. Without a trusted-proxy declaration, Gin trusts every proxy and `ClientIP()` returns the left-most `X-Forwarded-For` entry — an attacker-controlled value. The trusted-proxy call is therefore not optional.

IPv6 addresses are keyed by `/64` prefix. Each limiter caps the number of tracked keys; when the cap is reached, new keys share a fallback bucket so a flood of unique keys cannot exhaust memory. Idle buckets are garbage-collected after the TTL. Exceeded requests receive `429 Too Many Requests` with an accurate `Retry-After` and a body appropriate for the caller: JSON for HTMX and API requests, an HTML fragment for browser navigation.

5.3. Gzip Compression

The gzip middleware:

- Only compresses when `Accept-Encoding: gzip` is present.
- Skips WebSocket / SSE upgrades.
- Skips already-compressed types (`image/`, `video/`, `application/zip`, `font/woff2`, etc.).
- Sets `Vary: Accept-Encoding` for correct caching.
- Removes `Content-Length` since the body is now chunked.

5.4. HTTP Security Headers

The `SecurityHeaders` middleware sets defence-in-depth headers on every response. Full rationale and rollout plan in [docs/security_headers_cookies_csrf_jwt_scheme.adoc](#).

Header	Default
<code>Content-Security-Policy(-Report-Only)</code>	tailored to HTMX + Tailwind; report-only in dev
<code>Strict-Transport-Security</code>	disabled (<code>HSTS_MAX_AGE=0</code>)
<code>X-Frame-Options</code>	DENY
<code>X-Content-Type-Options</code>	nosniff
<code>Referrer-Policy</code>	strict-origin-when-cross-origin
<code>Permissions-Policy</code>	camera/mic/geo/etc. disabled
<code>Cross-Origin-Opener-Policy</code>	same-origin
<code>Cross-Origin-Resource-Policy</code>	same-origin

5.5. Secure Cookies

Three cookies are set, all with `Secure` (gated on `COOKIE_SECURE`), `SameSite=Lax`, and `Path=.`

Cookie	HttpOnly	Purpose
<code>auth_token</code>	yes	Short-lived access token (<code>JWT_ACCESS_TTL</code> , default 15 min)
<code>refresh_token</code>	yes	Long-lived refresh token (<code>JWT_REFRESH_TTL</code> , default 7 days)
<code>csrf_token</code>	no	Readable by JavaScript; required for the double-submit pattern

5.6. CSRF Protection

Every mutating request (`POST`, `PUT`, `PATCH`, `DELETE`) carries a CSRF token that the server compares against the `csrf_token` cookie using a constant-time comparison. In addition, the cookie value carries an HMAC signature; the middleware rejects any cookie whose signature does not verify under the server's `CSRF_SECRET`. On failure, the request is rejected with `403` before the handler runs.

Two paths carry the token:

- **Native forms** (login, register, user create/edit) embed it as `<input type="hidden" name="csrf_token">`.
- **HTMX requests** get it injected as the `X-CSRF-Token` header by an `htmx:configRequest` listener in `base.html`. No per-element handling is required; adding a new `hx-delete` button is automatically protected.

The CSRF token is rotated on login and register, alongside the JWT cookies.

Bearer-authenticated requests are exempt from CSRF validation. The exemption applies only when `AuthMiddleware` has validated the token and set the marker in the context; a request that merely carries an `Authorization` header is treated like any other request.

5.7. Authentication

The application uses a **two-token JWT scheme**:

- **Access token** — short-lived (15 min default), presented on every request. Stateless; not checked against any server-side store.
- **Refresh token** — long-lived (7 days default), carries a `jti` claim recorded in the `sessions` table. Exchanged for a fresh access token by the middleware when the access token expires.

Two transports are supported:

- **Cookie** (`auth_token`, `refresh_token`) — used by browser and HTMX requests.
- **Authorization: Bearer** `<access-token>` — used by API clients. Bearer requests are exempt from CSRF and are not eligible for silent refresh.

Bearer tokens are issued by `POST /api/v1/auth/token`, which accepts a JSON body with `email` and `password` and returns a short-lived access token. Cookie authentication is not accepted on `/api/*`: the API group has no CSRF middleware, so accepting cookies there would expose the JSON endpoints to CSRF.

Middleware behaviour on unauthenticated requests:

Client	Response
HTMX	401 + <code>HX-Redirect: /web/login</code>
API (<code>/api/*</code>)	401 JSON <code>{"error": "unauthorized"}</code>
Browser	302 to <code>/web/login</code>

5.7.1. Silent Refresh

When a request arrives with an expired `auth_token` but a valid, unrevoked `refresh_token`, the middleware reloads the user from the database, mints a fresh access token carrying the user's current role and activation status, sets it as a new `auth_token` cookie, and lets the request proceed. The user notices nothing.

A 15-minute access TTL combined with silent refresh means the practical session length is `JWT_REFRESH_TTL` (7 days), while the credential the

browser holds at any moment is only valid for 15 minutes.

Because the middleware reloads the user, a change made outside the application — a role edit via SQL, a deactivation — is reflected in the very next access token. It does not wait for the refresh TTL to expire.

5.7.2. Revocation

Logout, role changes, and account deactivation all revoke the corresponding `sessions` row. A revoked refresh token cannot be exchanged for an access token, so the session ends within the access-token TTL. This is what makes logout and privilege changes effective server-side, which a stateless JWT alone cannot provide.

5.8. Authorization

Three roles exist: `user`, `moderator`, and `admin`. Every route falls into one of three categories:

Category	Routes
Public — no authentication required	<code>/</code> , <code>/web/home</code> , <code>/web/login</code> , <code>/web/register</code> , <code>/web/refresh</code> , <code>/api/v1/auth/token</code> , <code>/health</code> , <code>/version</code> , <code>/static/*</code>
Authenticated — a valid session is required, any role	<code>/web/members</code> , <code>/web/members/partial</code> , <code>/web/me</code> , <code>/web/users/stats</code> , <code>/web/logout</code>
Admin-only — a valid session and <code>role=admin</code>	<code>/web/users</code> , <code>/web/users/by-id</code> , <code>/web/users/new</code> , <code>/web/users/:id/edit</code> , <code>/web/users/:id/role</code> , <code>/api/v1/users/*</code>

The category is enforced by which middleware is registered on the route group, not by checks inside the handler. `RequireRole` reads the claims set by `AuthMiddleware` and aborts with `403` (JSON for HTMX/API, or a redirect to `/web/me` for browser navigation) when the role does not match. Only the admin-only group is registered with `RequireRole`; the public and authenticated groups are not.

6. Versioning

The binary reports its own build metadata through two channels: the startup log line and the public `GET /version` endpoint.

6.1. Build-time injection

`scripts/build.sh` (and the `build` target in the Makefile) resolves four values from git and injects them via `-ldflags -X`:

Variable	Source	Example
Version	<code>git describe --tags --exact-match</code> if HEAD is a tag, else <code>git describe --tags --always --dirty</code>	<code>v0.8.1</code>
GitCommit	<code>git rev-parse --short=12 HEAD</code>	<code>9d4e6e335a06</code>
GitBranch	<code>git rev-parse --abbrev-ref HEAD</code>	<code>main</code>
BuildDate	<code>date -u +%Y-%m-%dT%H:%M:%SZ</code>	<code>2026-10-05T12:34:56Z</code>

A plain `go run .` or `go build` without `-ldflags` leaves these empty; `internal/version.Get()` then falls back to the VCS stamp that the Go toolchain embeds in every build taken from inside a git repository (available since Go 1.18). The commit hash, build time, and dirty flag are always present; only the semantic version and branch are missing.

6.2. Runtime query

```
curl -s http://localhost:8080/version | jq .
```

```
{
  "version": "v0.8.1",
  "commit": "9d4e6e335a06",
  "branch": "main",
```

```
"state": "v0.8.1",  
"built": "2026-10-05T12:34:56Z",  
"go": "1.23.0",  
"modified": false  
}
```

The endpoint is public and unauthenticated: it carries no user data and is used by operators, load balancers, and the release smoke test to identify which binary is running.

6.3. Release procedure

Releases follow Git Flow. Set the version prefix once:

```
git config --global gitflow.prefix.versiontag v
```

Then, from `develop`:

```
git flow release start 0.8.1  
# update the version lines in the AsciiDoc documents, add a  
# "What's New in v0.8.1" section, rebuild the PDFs, commit  
git flow release finish 0.8.1  
git push origin main  
git push origin develop  
git push origin v0.8.1
```

Do **not** run `git push --tags` — push the single release tag explicitly so unrelated local tags are not published.

7. Common Development Workflow

7.1. Daily Development

```
make dev
```

Runs Tailwind in watch mode in the background and the Go server under Air. Go files reload automatically; CSS rebuilds on template/style changes.

7.2. Database Changes

7.2.1. 1. Create a Migration

```
make migrate-create name=add_user_avatar
```

This creates two files in `migrations/`:

- `XXXX_add_user_avatar.up.sql` — forward migration
- `XXXX_add_user_avatar.down.sql` — rollback migration

Fill the `.up.sql` file with the forward SQL and the `.down.sql` file with the rollback SQL. Do not add tool markers (`-- +goose Up`, `-- +migrate Up`, etc.); `golang-migrate` reads the direction from the filename suffix, not from the file contents.

7.2.2. 2. Write the Migration

```
-- migrations/XXXX_add_user_avatar.up.sql
ALTER TABLE users ADD COLUMN avatar_url VARCHAR(500);
```

```
-- migrations/XXXX_add_user_avatar.down.sql
ALTER TABLE users DROP COLUMN avatar_url;
```

7.2.3. 3. Apply / Rollback

```
make migrate-up      # apply
make migrate-down    # rollback one
make db-reset        # full reset (drop, up, seed)
```

7.2.4. Troubleshooting a Dirty Migration

If `migrate up` fails partway, the `schema_migrations` row is marked dirty and further migrations refuse to run. Recover:

```
# Clear the dirty flag (and set the current version)
migrate -path migrations -database "$DB_URL" force 0

# Drop whatever the partial run left behind
psql -d mywebapp -c "DROP TABLE IF EXISTS sessions CASCADE; DROP TABLE IF EXISTS users CASCADE;"

# Reapply from scratch
make migrate-up
```

Or, more simply, drop the whole schema and reapply:

```
psql -d mywebapp -c "DROP SCHEMA public CASCADE; CREATE SCHEMA public;"
make migrate-up
make seed
```

7.3. Adding New Queries

7.3.1. 1. Add SQL Queries

Add queries to `sqlc/queries.sql` (or a new file in `sqlc/`, if `sqlc.yaml` lists it):

```
-- name: GetUserByEmailAndActive :one
SELECT id, email, username, full_name, is_active, role, last_login, created_at, updated_at
FROM users
WHERE email = $1 AND is_active = $2
LIMIT 1;

-- name: CountUsersByRole :one
SELECT COUNT(*) FROM users WHERE role = $1;
```

7.3.2. 2. Regenerate SQLC Code

```
make generate
```

7.3.3. 3. Use the Generated Code

```
user, err := s.queries.GetUserByEmailAndActive(ctx, "user@example.com", true)
count, err := s.queries.CountUsersByRole(ctx, pgtype.Text{String: "admin", Valid: true})
```

7.4. Adding a New API Endpoint

7.4.1. 1. Update Domain Models

Add DTOs in `internal/domain/your_model.go`:

```
type CreatePostRequest struct {
    Title  string `json:"title" binding:"required"`
    Content string `json:"content" binding:"required"`
}

type PostResponse struct {
    ID        int32    `json:"id"`
    Title     string   `json:"title"`
    Content   string   `json:"content"`
    CreatedAt time.Time `json:"created_at"`
}
```

7.4.2. 2. Add Service Method

```
func (s *PostService) CreatePost(ctx context.Context, req domain.CreatePostRequest) (*domain.Post, error) {
    post, err := s.queries.CreatePost(ctx, repository.CreatePostParams{
        Title:  req.Title,
        Content: req.Content,
    })
    if err != nil {
        return nil, fmt.Errorf("create post: %w", err)
    }
    return s.mapCreatePostRowToDomain(&post), nil
}
```

7.4.3. 3. Add Handler

Remember to pull the request-scoped logger for any error paths:

```
func (h *PostHandler) CreatePost(c *gin.Context) {
    logger := middleware.LoggerFromGin(c)

    var req domain.CreatePostRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    post, err := h.service.CreatePost(c.Request.Context(), req)
    if err != nil {
        logger.Error("api: CreatePost failed", "error", err)
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Something went wrong."})
        return
    }

    c.JSON(http.StatusCreated, domain.ToPostResponse(post))
}
```

7.4.4. 4. Register Route

```
api := router.Group("/api/v1",
    authMW,
    middleware.RequireRole("admin"),
)
{
    posts := api.Group("/posts")
    {
```

```
posts.POST("", postHandler.CreatePost)
posts.GET("", postHandler.ListPosts)
posts.GET("/:id", postHandler.GetPost)
}
}
```

7.5. Adding a New HTMX Page

7.5.1. 1. Add a Handler

```
func (h *WebHandler) ListPostsPage(c *gin.Context) {
    posts, _ := h.service.ListPosts(c.Request.Context(), 20, 0)

    data := gin.H{
        "posts":      posts,
        "csrf_token": middleware.TokenFromContext(c),
    }

    if IsHTMX(c) {
        c.HTML(http.StatusOK, "post_list.html", data)
        return
    }
    c.HTML(http.StatusOK, "base.html", gin.H{
        "title":      "Posts",
        "listEndpoint": "/web/posts",
        "csrf_token":  middleware.TokenFromContext(c),
    })
}
```



Pass `csrf_token` into the template data for any page that renders a mutating form. HTMX requests get the token injected automatically by the `htmx:configRequest` listener, but native form submissions need the hidden field.

7.5.2. 2. Add Templates

Following the existing pattern:

- `internal/templates/partials/post_list.html` — the HTMX partial
- Optionally `internal/templates/partials/post_row.html` — one row

7.5.3. 3. Register the Route

```
webGroup := router.Group("/web",
    authMW,
    middleware.RequireRole("admin"),
    middleware.CSRFMiddleware(csrfCfg),
)
webGroup.GET("/posts", webHandler.ListPostsPage)
```

8. Project Structure

```
mywebapp/
├── bin/                # Compiled binaries (gitignored)
├── cmd/
│   └── api/
│       └── main.go    # Entry point, middleware wiring, session cleanup
├── docs/
│   ├── images/
│   │   ├── webapp_dataflow_2.puml
│   │   ├── webapp_dataflow_2.svg
│   │   ├── webapp_sequence_login.svg
│   │   └── scc_report_v0.8.1.svg
│   ├── security_headers_cookies_csrf_jwt_scheme.adoc
│   └── security_headers_cookies_csrf_jwt_scheme.pdf
├── internal/
│   ├── auth/
│   │   ├── jwt.go      # Access + refresh token generation and validation
│   │   ├── jwt_test.go
│   │   └── password.go  # bcrypt hashing
│   ├── domain/
│   │   ├── errors.go   # Domain-level sentinel errors
│   │   └── user.go      # Models and DTOs
│   ├── handler/
│   │   ├── user_handler.go    # JSON API handlers
│   │   └── user_handler_test.go
│   ├── logging/
│   │   └── logging.go         # slog setup, Gin writer bridge
│   ├── middleware/
│   │   ├── accesslog.go
│   │   ├── accesslog_test.go
│   │   ├── csrf.go          # Signed double-submit cookie pattern
│   │   └── csrf_test.go
```



```

├── login_form.html
├── members_list.html    # Member directory HTMX fragment
├── members_row.html     # Member directory row (read-only)
├── register_form.html
├── user_form.html
├── user_list.html
├── user_list_by_id.html
├── user_role_form.html
├── user_row.html
├── user_stats.html
├── testutil/
│   ├── db.go            # Test pool bootstrap and schema reset
│   ├── migrate.go       # In-process migration runner
│   ├── paths.go         # Repository-root resolver
│   └── seed.go          # Demo users for tests
├── version/
│   └── version.go       # Build metadata via -ldflags -X
└── web/
    ├── auth_handler.go  # Login / Register / Logout / Refresh / API token
    ├── auth_handler_test.go
    ├── auth_middleware.go # JWT middleware + silent refresh
    ├── auth_middleware_test.go
    ├── cookies.go
    ├── cookies_test.go
    ├── health_handler.go # /health and /version
    ├── helpers.go
    ├── helpers_test.go
    ├── home_handler.go  # /web/home, /web/members
    ├── home_handler_test.go
    ├── main_test.go     # Package-level TestMain
    ├── me_handler.go    # /web/me
    ├── me_handler_test.go
    ├── template_funcs.go
    └── template_funcs_test.go

```

```

├── web_handler.go      # Admin HTMX page handlers
├── web_handler_test.go
├── logs/
│   ├── app.log        # Structured logs (gitignored)
├── migrations/
│   ├── 20260906221500_create_users_table.up.sql
│   ├── 20260906221500_create_users_table.down.sql
│   ├── 20260922120000_create_sessions_table.up.sql
│   └── 20260922120000_create_sessions_table.down.sql
├── pkg/
│   ├── db/
│   │   ├── db.go      # Pool config, .env loading
│   │   └── tx.go      # WithTx helper
│   └── r2/
│       └── client.go   # Cloudflare R2 presigned URLs
├── scripts/
│   ├── build.sh       # Release build with -ldflags -X
│   └── seed.go        # Demo data seeding
├── sqlc/
│   ├── queries.sql
│   ├── queries_sessions.sql
│   ├── sqlc.yaml
│   └── sqlc.yml -> sqlc.yaml
├── static/
│   ├── css/
│   │   ├── output.css # Tailwind build output
│   │   └── tailwind.css
│   └── js/
│       └── htmx.min.js
├── .env               # Environment (gitignored)
├── .air.toml          # Air configuration
├── .gitignore
├── Makefile           # Build automation
├── go.mod

```

- go.sum
- package.json
- tailwind.config.js
- README.adoc

This file

9. Project inventory for v0.8.1

Source Code Statistics — MyWebApp v0.8.1

Generated by scc · 115 files · 162,344 lines of code

Language	Files	Lines	Blank	Comment	Code	Complexity	Bytes	Uloc
Go	67	11,677	1,544	1,264	8,869	1,661	331,581	0
HTML	16	1,869	131	28	1,710	0	92,900	0
Shell	10	933	127	138	668	54	30,630	0
SQL	6	190	28	26	136	6	6,026	0
SVG	4	914	160	158	596	2	46,879	0
AsciiDoc	3	4,670	979	0	3,691	0	153,909	0
CSS	2	52	1	0	51	0	28,561	0
JavaScript	2	27	0	1	26	539	48,325	0
YAML	2	39	2	1	36	0	685	0
JSON	1	15	0	0	15	0	520	0
Makefile	1	529	70	56	403	46	19,896	0
TOML	1	38	4	1	33	0	726	0
Total	115	20,953	3,046	1,673	16,234	2,308	760,638	0

COCOMO Estimates — Organic

Estimated Cost to Develop	\$504,183
Estimated Schedule Effort	10.60 months
Estimated People Required	4.22

Total Lines (incl. blank & comments)	20,953
Lines of Code (excl. blank & comments)	16,234
Complexity Score (total)	2,308

Figure 2. Source code statistics for MyWebApp v0.8.1

10. Available Make Commands

10.1. Application

Command	Description
<code>make run</code>	Run the application in the foreground
<code>make start</code>	Build and start in the background (writes <code>.pid</code>)
<code>make stop</code>	Stop the background process (SIGTERM, then SIGKILL after 10s)
<code>make restart</code>	Stop + start
<code>make status</code>	Show whether the app is running
<code>make logs</code>	Tail <code>logs/app.log</code>
<code>make dev</code>	Live reload (Air) + Tailwind watch
<code>make build</code>	Build Tailwind + production binary (with version metadata)
<code>make build-css</code>	Build Tailwind CSS only
<code>make watch-css</code>	Tailwind in watch mode
<code>make version</code>	Print the version metadata that <code>make build</code> will inject
<code>make version-run</code>	Query <code>/version</code> on the running server

10.2. Dependencies

Command	Description
<code>make install</code>	Go + Node + dev tools
<code>make generate</code>	Regenerate SQLC code

10.3. Database

Command	Description
<code>make migrate-up</code>	Apply migrations
<code>make migrate-down</code>	Roll back one migration
<code>make migrate-create name=<name></code>	Create a new migration pair
<code>make seed</code>	Seed demo users (bcrypt-hashed)
<code>make db-reset</code>	stop → migrate-down → migrate-up → seed
<code>make db-shell</code>	Open <code>psql</code>
<code>make db-check</code>	Verify connection
<code>make db-tables</code>	List tables

10.4. Test database

Command	Description
<code>make test-db-create</code>	Create the test database (idempotent)
<code>make test-db-drop</code>	Drop the test database (with confirmation)
<code>make test-db-reset</code>	Drop and recreate the test database
<code>make test-db-check</code>	Print the resolved test configuration
<code>make test-db-shell</code>	Open <code>psql</code> against the test database

10.5. Testing & Quality

Command	Description
---------	-------------

<code>make test</code>	Run all tests (serialised; uses <code>TEST_DB_*</code>)
<code>make test-fast</code>	Run non-DB packages in parallel
<code>make test-coverage</code>	Raw HTML coverage over every package
<code>make test-coverage-src</code>	Source-only coverage, excludes wiring and generated packages
<code>make fmt</code>	<code>go fmt</code> + CSS rebuild
<code>make lint</code>	<code>golangci-lint</code> (auto-installs if missing)

10.6. Docker

Command	Description
<code>make docker-up</code>	Start containers
<code>make docker-down</code>	Stop containers
<code>make docker-build</code>	Build images
<code>make docker-logs</code>	Follow container logs

10.7. Setup & Utilities

Command	Description
<code>make setup</code>	install → generate → migrate-up → build-css
<code>make dev-setup</code>	Same as <code>setup</code> , hints at <code>make dev</code>
<code>make all</code>	clean → install → generate → migrate-up → build-css → build
<code>make clean</code>	Remove binaries, <code>node_modules</code> , CSS output
<code>make check-env</code>	Print loaded environment
<code>make info</code>	Project metadata (includes version, commit, branch)

make dirs	Create standard directory layout
-----------	----------------------------------

11. API Endpoints

11.1. Authentication API

Method	Endpoint	Description
POST	<code>/api/v1/auth/token</code>	Exchange credentials for a Bearer access token

The token endpoint is public and rate-limited. It returns `{"access_token": "...", "token_type": "Bearer", "expires_in": 900}`.

11.2. Users API

Method	Endpoint	Description
POST	<code>/api/v1/users</code>	Create a new user
GET	<code>/api/v1/users</code>	List users (paginated)
GET	<code>/api/v1/users/:id</code>	Get user by ID
PUT	<code>/api/v1/users/:id</code>	Update user
PUT	<code>/api/v1/users/:id/role</code>	Change a user's role
DELETE	<code>/api/v1/users/:id</code>	Delete user

All Users API routes require authentication via `Authorization: Bearer <access-token>` and require `role=admin`. Cookie authentication is refused on `/api/*`.

11.3. Web (HTMX) Routes

Method	Endpoint	Description
--------	----------	-------------

GET	/	Public home page (no auth required)
GET	/web/home	Public home page (explicit URL)
GET	/web/members	Member directory (authenticated, any role)
GET	/web/members/partial	Member directory HTMX fragment
GET	/web/login	Login page / form partial
POST	/web/login	Submit login
GET	/web/register	Register page / form partial
POST	/web/register	Submit registration
POST	/web/refresh	Exchange refresh token for a new access token
POST	/web/logout	Revoke the session and clear all cookies
GET	/web/me	Self-service account page (any authenticated role)
GET	/web/users	Users list (admin)
GET	/web/users/by-id	Users list by ID (admin)
GET	/web/users/new	New user form (admin)
POST	/web/users	Create user (admin)
GET	/web/users/:id/edit	Edit user form (admin)
PUT	/web/users/:id	Update user (admin)
GET	/web/users/:id/role	Role-change form (admin)

PUT	/web/users/:id/role	Submit role change (admin)
DELETE	/web/users/:id	Delete user (admin)
GET	/web/users/stats	Live member counts (any authenticated role)

11.4. Operational Endpoints

Method	Endpoint	Description
GET	/health	DB ping + pgx pool statistics
GET	/version	Build metadata as JSON
GET	/test	Template smoke test
GET	/static/*	Static assets

11.5. Example Requests

11.5.1. API token (JSON)

```
POST /api/v1/auth/token HTTP/1.1
Content-Type: application/json

{
  "email": "alice@example.com",
  "password": "password123"
}
```

Response:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 900
}
```

11.5.2. Login (HTMX)

```
POST /web/login HTTP/1.1
Content-Type: application/x-www-form-urlencoded

csrf_token=<token>&email=alice@example.com&password=password123
```

Response sets `auth_token`, `refresh_token`, and a rotated `csrf_token`, and returns `HX-Redirect: /web/home` for HTMX clients.

11.5.3. Refresh (browser / HTMX)

```
POST /web/refresh HTTP/1.1
Cookie: refresh_token=<jwt>
```

Response is **204 No Content** with a new **Set-Cookie: auth_token=...** header. On failure (expired, revoked, or missing refresh token), returns **401** and clears both auth cookies.

11.5.4. Logout (HTMX)

```
POST /web/logout HTTP/1.1
X-CSRF-Token: <token>
Cookie: auth_token=<jwt>; refresh_token=<jwt>; csrf_token=<token>
```

Response revokes the session row, clears all three cookies, and returns **HX-Redirect: /web/login**.

11.5.5. Create User (JSON API)

```
POST /api/v1/users HTTP/1.1
Content-Type: application/json
Authorization: Bearer <access-token>

{
  "email": "john@example.com",
  "password": "securepassword123",
  "username": "johndoe",
  "full_name": "John Doe"
}
```

Response:

```
{
  "id": 6,
  "email": "john@example.com",
  "username": "johndoe",
  "full_name": "John Doe",
  "is_active": true,
  "role": "user",
  "created_at": "2026-10-05T12:34:56Z",
  "updated_at": "2026-10-05T12:34:56Z"
}
```



The `role` field is not accepted on create or update. New users are always created with `role=user`. Role changes go through `PUT /api/v1/users/:id/role`.

11.5.6. List Users (paginated)

```
GET /api/v1/users?page=1&limit=10 HTTP/1.1
Authorization: Bearer <access-token>
```

Response:

```
{
  "data": [
    {
      "id": 1,
      "email": "alice@example.com",
      "username": "alice",
      "full_name": "Alice Johnson",
      "is_active": true,
      "role": "admin",
      "created_at": "2026-09-06T22:15:00Z",

```

```
        "updated_at": "2026-09-22T20:43:53Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 10,
      "total": 5
    }
  }
}
```

11.5.7. Health Check

GET /health HTTP/1.1

Response:

```
{
  "status": "ok",
  "time": 1758567833,
  "pool": {
    "total": 5,
    "idle": 5,
    "acquired": 0,
    "max": 25,
    "constructing": 0,
    "acquire_count": 12,
    "empty_acquire": 0,
    "canceled_acquire": 0,
    "acquire_duration_ms": 3
  }
}
```

11.5.8. Version

```
GET /version HTTP/1.1
```

Response:

```
{  
  "version": "v0.8.1",  
  "commit": "9d4e6e335a06",  
  "branch": "main",  
  "state": "v0.8.1",  
  "built": "2026-10-05T12:34:56Z",  
  "go": "1.23.0",  
  "modified": false  
}
```

12. Troubleshooting

12.1. Common Issues

12.1.1. Database Connection Failed

Check `.env` and confirm PostgreSQL is running:

```
pg_isready  
make check-env  
make db-check
```

12.1.2. SQLC Generation Fails

```
cd sqlc && sqlc generate  
# or:  
make generate
```

Verify no syntax errors in `sqlc/queries.sql` and that `sqlc.yaml` lists both `queries.sql` and `queries_sessions.sql`.

12.1.3. Migration Fails

```
# Inspect current version  
migrate -path migrations -database "$DB_URL" version  
  
# Force a version if a partial migration left the schema inconsistent  
migrate -path migrations -database "$DB_URL" force <version>
```

For a development database, dropping and recreating the schema is usually faster:

```
psql -d mywebapp -c "DROP SCHEMA public CASCADE; CREATE SCHEMA public;"
make migrate-up
make seed
```

12.1.4. migrate: command not found

The `migrate` binary is installed by `go install` into `$(go env GOPATH)/bin`. If that directory is not on your `PATH`:

```
# One-time, per shell
export PATH="$HOME/go/bin:$PATH"

# Or make it permanent
echo 'export PATH="$HOME/go/bin:$PATH"' >> ~/.zshrc # or ~/.bashrc
```

12.1.5. unknown driver postgres (forgotten import?)

The `migrate` binary was installed without the PostgreSQL driver. Reinstall with the build tag:

```
go install -tags 'postgres' github.com/golang-migrate/migrate/v4/cmd/migrate@latest
```

12.1.6. no migration found for version 0

The `schema_migrations` table has a phantom version-0 row left by `migrate force 0`. Clear it:

```
psql -d mywebapp -c "DELETE FROM schema_migrations;"
make migrate-up
```

12.1.7. Air Not Found

```
make install
# or:
go install github.com/air-verse/air@latest
```

12.1.8. Rate Limited During Development

If HTMX polling (`/web/users/stats`) trips the global limiter, either:

- Reduce polling frequency in `base.html`.
- Increase the global limiter in `cmd/api/main.go`:

```
globalLimiter := middleware.NewRateLimiter(60, 120, 10*time.Minute, 100_000)
```

12.1.9. Logged Out Immediately After Login

Two causes:

1. `COOKIE_SECURE=true` while running over plain HTTP. The browser accepts the cookie on the login response but refuses to send it back on the next request. Fix: set `COOKIE_SECURE=false` in `.env` for local dev.
2. The `auth_token` cookie's `Max-Age` is shorter than the time between login and the next request. With the 15-minute default, this is unlikely unless `JWT_ACCESS_TTL` was set to a very short value.

12.1.10. Chrome Serves a Stale Redirect for /

If `http://localhost:8080/` redirects to `/web/login` in Chrome but not in Firefox, the browser has cached a `302` from an earlier build. The current server emits `Cache-Control: no-store` on that route, but a cached entry from before the current build can survive a redeploy.

Clear it: DevTools → Application → Storage → **Clear site data**, or `chrome://settings/clearBrowserData` with **Cached images and files**

Cookies and other site data, All time.

The `NoCache` middleware prevents recurrence for any response emitted through the middleware chain. `abortUnauthenticated` additionally sets `no-store` on 401s and login redirects so no intermediary can cache them.

12.1.11. `IS_AUTHED = false` on `/web/home`

If the Home page renders the anonymous variant while you are in fact signed in, the server did not see a valid access token when it rendered. Possible causes:

1. The access token has expired and `OptionalAuthMiddleware` does not perform a silent refresh. The next protected request will refresh it; a reload of `/web/home` after that will render authenticated.
2. `COOKIE_SECURE=true` on plain HTTP — the browser never stores the auth cookie, so no token is present on any request.
3. The browser is serving a stale copy. Clear site data for `localhost` and reload.

You can confirm the server's view directly:

```
curl -s -b /tmp/cj.txt http://localhost:8080/web/home \  
| grep -o 'const IS_AUTHED = [a-z]*'
```

`true` means the server rendered the authenticated variant; a browser still showing the anonymous variant is serving stale content.

12.1.12. Silent Refresh Fails in One Browser but Not Another

Check the browser console for errors on the `/web/refresh` request. The most common cause is a stale `refresh_token` cookie that was set with different attributes than the current request expects — clear all cookies for `localhost:8080` and log in again.

12.1.13. CSRF 403 on a Form Submission

The form's hidden `csrf_token` field is empty, or doesn't match the cookie. Reload the page (which re-issues the cookie via the safe-method branch) and resubmit. If it recurs, verify that `middleware.CSRFMiddleware` is registered on the `GET` route that renders the form.

If the `CSRF_SECRET` env var is not set and the process has just restarted, all outstanding tokens become invalid because the per-process key changed. Reload the page. In a cluster, set `CSRF_SECRET` to the same value on every node.

12.1.14. DB-dependent tests skip

Symptom:

```
testutil: no test database configured; DB-dependent tests will skip
--- SKIP: TestCreateUser_AdminSucceeds
```

Cause: `TEST_DB_NAME` is empty, or the test database does not exist.

Fix:

```
# Confirm the configuration is loaded.
make test-db-check

# If TEST_DB_NAME is set but the database does not exist:
make test-db-create
```

12.1.15. Test database unreachable

Symptom:

```
testutil: test database unreachable: ... password authentication failed for user "pmjtoca"
```

Cause: `TEST_DB_PASSWORD` in `.env` does not match the current password for the `TEST_DB_USER` role, or the role does not exist.

Fix: update `TEST_DB_PASSWORD` in `.env`, or create the role. If the project uses SCRAM authentication (see the security document), the password is enforced and must be correct.

12.1.16. multiple definitions of TestMain

Symptom:

```
# mywebapp/internal/web  
multiple definitions of TestMain  
FAIL    mywebapp/internal/web [setup failed]
```

Cause: two files in the same test package define `TestMain`. Go allows exactly one.

Fix: keep the definition in `main_test.go` and remove any other. Search with:

```
grep -rn 'func TestMain' internal/
```

Only one match per package is correct.

12.1.17. Logs Are JSON When I Want Text

Set explicitly in `.env`:

```
LOG_FORMAT=text
```

12.1.18. I Want Debug-Level Logs

```
LOG_LEVEL=debug
```

Debug mode also includes `source` (file:line) on every line.

12.2. Getting Help

- Tail the structured logs: `make logs`
- Grep one request: `grep '"request_id":"' logs/app.log`
- Check the health endpoint: `curl localhost:8080/health | jq`
- Check the running version: `curl localhost:8080/version | jq`
- Inspect active sessions:

```
psql -d mywebapp -c "SELECT jti, user_id, issued_at, expires_at, revoked_at FROM sessions ORDER BY id DESC LIMIT 10;"
```

- Official docs:
 - <https://gin-gonic.com/>
 - <https://htmx.org/>
 - <https://sqlc.dev/>
 - <https://github.com/golang-migrate/migrate>
 - <https://github.com/air-verse/air>
 - <https://pkg.go.dev/log/slog>
 - <https://github.com/golang-jwt/jwt>

13. Contributing

1. Fork the repository
2. Create a feature branch
3. Make your changes

4. Run `make test` and `make lint`
5. Submit a pull request

13.1. Conventions

- Use `middleware.LoggerFromGin(c)` (not `slog.Default()`) inside handlers.
- Return domain sentinel errors (`domain.ErrUserNotFound`, etc.) from services; map them in handlers.
- New SQL goes in `sqlc/queries.sql` or `sqlc/queries_sessions.sql`; never hand-edit `internal/repository/*.go`.
- New HTMX pages should expose a partial template and use a stable container id (e.g. `#user-list`, `#member-list`).
- Pass `csrf_token` into template data for any page rendering a mutating form.
- Mutating HTMX requests get `X-CSRF-Token` injected automatically — no per-element handling needed.
- The root route `/` and `/web/home` are public. Any new protected route belongs behind `authMW`, and any admin-only route behind `RequireRole("admin")`.
- The public Home page uses `OptionalAuthMiddleware`. Do not swap in `authMW` on that route — it would break anonymous access.
- New DB-dependent tests should call `testutil.NewPool(t)`, never `db.LoadConfig()` or `db.NewPool()`. The production database must not be reachable from the test code path.

14. License

This project is licensed under the MIT License — see the LICENSE file for details.

15. Contact

- Author: patrick toca
- Email: pmjtoca@gmail.com
- Issue Tracker: <https://github.com/yourusername/mywebapp/issues>

- Repository: <https://github.com/yourusername/mywebapp>